SCHEDLAB: A RESEARCH SHELL FOR JOB SCHEDULING EXPERIMENTS

Isobella Gordon, Patrick Araujo, Xueyan Zhang

Northeastern University

CS5600 – Computer Systems

Prof. Jack Thomas

October 15, 2026

I. PROJECT TITLE & TEAM 2
II. ABSTRACT 2
III. THE RESEARCH EXTENSION 2
IV. RESEARCH QUESTION 3
V. PROPOSED SCOPE 4

I. PROJECT TITLE & TEAM

Title: SchedLab: A Research Shell for Job Scheduling Experiments

Subtitle: Comparing FIFO, SJF, Priority, and Adaptive Scheduling Policies in a UNIX Shell

Name	Email	Role
Isobella Gordon	gordon.i@northeastern.edu	Team Bio, Exploration Report
Patrick Araujo	araujo.pa@northeastern.edu	Shell implementation, documentation
Xueyan Zhang	zhang.xueya@northeastern.edu	GitHub setup, UML diagrams, screenshots

II. ABSTRACT

Modern UNIX shells (bash, zsh, fish) start every background job immediately, with no limit on how many run at once. When many CPU-heavy jobs run, the terminal becomes slow. In our tests on an Ubuntu VM, running 10 background jobs made a simple ls command go from 4ms to 569ms - about 142x slower. SchedLab is a UNIX shell that fixes this by limiting how many background jobs run at the same time and choosing which queued job starts next. It supports four policies: FIFO, Shortest Job First (SJF), Priority, and Adaptive. We measure foreground command latency, background throughput, and fairness. Our goal is to see if a simple concurrency limit can keep the terminal responsive without losing too much background speed.

III. THE RESEARCH EXTENSION

Core idea: The shell should decide when to start each background job and how many to run at once, instead of starting everything immediately.

Two dimensions:

1. **Concurrency limit (N):** How many background jobs run at the same time.
    a. setconcurrency N - the user sets the limit.
    b. Jobs beyond the limit wait in a queue.

2. **Scheduling policy:** Which queued job starts next when a slot opens.

Policy	How it works	Use case
FIFO	Min-Heap / Queue ordered by arrival timestamp (O(1) enqueue/dequeue)	Simple, predictable

SJF	Min-Heap ordered by predicted runtime using an Exponential Moving Average (EMA) history table.	Minimize average wait time
Priority	Min-Heap ordered by user-defined nice priority (with aging to prevent starvation) paired with OS setpriority().	Prioritize important jobs
Adaptive	AIMD feedback controller that adjusts the concurrency limit N dynamically based on foreground latency.	Self-tuning

Hypothesis:

An adaptive policy that adjusts the concurrency limit at runtime will match the foreground latency of the best fixed limit, without the user having to tune it manually. Among the fixed policies, SJF will give the lowest average wait time, and Priority will be best when jobs have different importance. We expect that a small limit (like the number of CPU cores) will dramatically improve foreground latency with only a small loss in background throughput.
Why this is new: No existing shell lets users limit concurrent background jobs or choose a scheduling policy. This is not a settings change - it is a new scheduling subsystem inside the shell.

IV. RESEARCH QUESTION

Can a UNIX shell that limits concurrent background jobs reduce foreground command latency under heavy load, compared to the unlimited execution used by bash, zsh, and fish?
Secondary questions:

1. How does the concurrency limit affect foreground latency?
2. How much background throughput is lost by limiting concurrency?
3. Does SJF reduce average wait time compared to FIFO?
4. Can the Adaptive policy match the best fixed limit without manual tuning?
5. How do the policies affect fairness between jobs?

Metrics:

Metric	What it measures	Unit
Foreground latency	Time to run a foreground command (ls) while background jobs run	ms

Background throughput	Jobs completed per second	jobs/s
Average wait time	Average time a job spends in the queue	s
Fairness (F)*	Jain's Fairness Index across n jobs, where x_i is job i's normalized turnaround slowdown	0 to 1

$$F = \frac{(\sum x_i)^2}{n \cdot \sum x_i^2}$$

Target: Reduce foreground latency by at least 10x (from ~500ms to under 50ms) while keeping background throughput within 20% of bash.

V. PROPOSED SCOPE

Task 1: Base shell

Implement a UNIX shell with:
Command parsing (quotes, variables)
Built-ins: cd, exit, pwd, export
Pipes (|) and redirects (>, <, >>, 2>)
Background jobs (&), job control (jobs, fg, bg, wait)
Signal handling (Ctrl+C, Ctrl+Z, SIGCHLD)

Task 2: Job queue with concurrency limit

A queue holds jobs waiting to start.
setconcurrency N controls how many run at once.
When a job finishes (SIGCHLD), the shell takes the next job from the queue.

Task 3: Three static policies

FIFO: queue in arrival order.
SJF: the shell remembers how long each command took last time and uses that to order the queue.
Priority: the user sets a priority with nice; the queue is sorted by priority.

Task 4: Adaptive policy

setlatency <ms> sets a target for foreground latency.
If the foreground is slower than the target, the shell lowers the concurrency limit.
If the foreground is faster than the target, the shell raises the limit.
Lightweight AIMD (Additive Increase, Multiplicative Decrease) feedback loop, inspired by TCP congestion control.

Task 5: Measurements and experiments

Log foreground latency, throughput, and wait times to a CSV file.
stats command shows the numbers in the terminal.
Experiments: 5, 10, and 20 jobs; limits of 1, 2, 3, 5, and unlimited; all four policies.
Compare with bash as baseline

VI. REFERENCES

Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2023). Operating Systems: Three Easy Pieces (Version 1.10). Arpaci-Dusseau Books. Available at: https://pages.cs.wisc.edu/~remzi/OSTEP/ (Chapters 5-8: Process API, Direct Execution, and CPU Scheduling: FIFO, SJF, and Priority).

Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2024). OSTEP Projects: Processes and UNIX Shell (processes-shell). GitHub Repository. Available at: https://github.com/remzi-arpacidusseau/ostep-projects

Ramey, C., & Fox, B. (2024). Bash Reference Manual: Job Control and Process Execution (Edition 5.2). Free Software Foundation. Available at: https://www.gnu.org/software/bash/manual/

Jain, R., Chiu, D. M., & Hawe, W. R. (1984). A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems. DEC Research Report TR-301.

Chiu, D. M., & Jain, R. (1989). Analysis of the Increase and Decrease Algorithms for Congestion Avoidance in Computer Networks (AIMD Feedback Control). Computer Networks and ISDN Systems, 17(1), 1-14.